L46 Project

Exploring Quantisation In FPGA-Accelerated Binarized Neural Networks

Alex Chau (*ac2822*), Churchill College

1 Introduction

The rapid advancement of machine learning has driven an increasing demand for efficient hardware acceleration of neural networks. While general-purpose processors such as CPUs and GPUs have traditionally been used for this purpose, significant performance and energy efficiency gains have emerged from the use of specialised hardware accelerators. In particular, field-programmable gate arrays (FPGAs) have gained attention due to their ability to provide flexible, fine-grained parallelism and custom dataflow architectures tailored to specific workloads.

A key technique enabling these improvements is quantisation, which reduces the numerical precision of neural network parameters and activations. By representing values with fewer bits, ranging from reduced fixed-point precision to fully binarized representations, quantisation significantly lowers memory usage and computational complexity. This, in turn, allows for faster inference, reduced power consumption, and more efficient utilisation of hardware resources. In extreme cases such as binarized neural networks (BNNs), arithmetic operations can be simplified to bitwise operations, making them especially well-suited for FPGA implementations.

Much of the recent progress in the AI hardware acceleration space can be attributed to the combination of increased hardware specialisation and aggressive quantisation strategies. Specialised architectures are able to exploit the reduced precision of quantised models to maximise parallelism and throughput, achieving performance levels that would be difficult to attain with conventional high precision designs.

This project explores the role of quantisation in FPGA-accelerated neural networks, with a particular focus on binarized neural networks. Using the FINN framework, quantized models are mapped to dataflow-style hardware architectures on an FPGA platform. The study investigates how neural networks trained in Brevitas are mapped to FPGA-accelerated neural networks and how different levels of quantisation that impact performance, as well as how these effects interact with architectural parallelism. Through these experiments, the project aims to provide insight into the trade-offs between accuracy, resource utilisation, and performance in quantised neural network deployment on reconfigurable hardware.

2 Experiment 1: Analysis of the PyTorch to FPGA transformation pipeline for Quantized Neural Networks

2.1 System Setup

To evaluate quantized neural network acceleration on FPGA hardware, the FINN framework was deployed on a Diligent Arty-Z7-20 development board. FINN is an open-source framework developed by Xilinx/AMD that enables deployment of quantized neural networkks (QNNs) on FPGA platforms through automated hardware generation and compilation.

2.1.1 Hardware Platform

The target platform used in this project is the Arty-Z7-20 development board, this board includes an Artix-7 FPGA as the programmable logic (PL) and an ARM Cortex-A9 processor in the form of a Zynq-7000 SOC as the programmable system (PL).

2.1.2 Software Environment

- PYNQ - A pre-built Linux image for Zynq SOCs (which interfaces with PL using Python) was flashed onto a microSD card and inserted onto the board.
  - There are a small number of pre-built images which AMD maintains, the Arty-Z7-20 isn't one of these boards. Luckily, one of the supported boards (Pynq-Z1), happens to have the exact same specs and (almost the same) pinouts as the Arty-Z7-20, making PYNQ images built for the Pynq-Z1 compatible with the Z7-20, we use the PYNQ v3.0.1 image in this work
- FINN - A compiler which maps Quantized Neural Networks (QNNs) trained in Brevitas to dataflow-style FPGA architectures [1].
- Xilinx Design Suite 2022.2

Figure 1: Image of the Arty-Z7-20

The rest of this subsection will detail the steps to replicate the environment for training quantized neural networks using Brevitas and converting it to a FPGA-deployable bitstream using the FINN compiler.

2.1.3 Ubuntu 24.04

Ubuntu 24.04 was selected as the development OS for a couple of reasons: The FINN compiler has dependencies better suited to UNIX-like operating systems, a NVIDIA GPU (RTX 3090) was the GPU available for training quantized neural networks, and Vivado tooling is hard to setup with FINN in a non-Unix environment. Ubuntu was installed following the interactive installer, one-thing to note is that the Vivado install uses a large amount of data in the /tmp directory. Some Ubuntu install guides recommend allocating a separate 20-30GB partition purely for /tmp, we recommended allocating only a single root partition for the install so that tmp doesn't run out of memory during the Vivado Design Suite Installation, as it uses over 80GB of /tmp memory.

2.1.4 Xilinx Design Suite 2022.2

The FINN compiler currently only supports Vivado Design Suite 2022.2, this is slightly troublesome as the newest version of Ubuntu this Vivado version supports is Ubuntu 22.04. Specifically, the `libtinfo55`, `libncurses5` and `libncurses5-dev` libraries are missing in Ubuntu 24.04, which causes the Vivado install to hang To install them we do the following:

```
sudo nano /etc/apt/sources.list.d/ubuntu.sources
```

At the end of the file, append:

```
Types: deb
URIs: http://old-releases.ubuntu.com/ubuntu/
Suites: lunar
Components: universe
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

Then save the file and run:

```
sudo apt update
```

Then install the required libraries:

```
sudo apt install libtinfo5 libncurses5-dev libncurses5
```

Now we can go to the Xilinx Download Page and download the unified installer for version 2022.2.

2.1.5 FINN

The FINN compiler requires a Docker install to run without root permissions:

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyr
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

Then to install the required dependencies:

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin d
```

To run docker rootless, we need to create a `docker` user group and add it to the current user.

```
sudo groupadd docker
sudo usermod -aG docker $USER
```

Logging out and logging back in should update the usergroups for the current user, we can confirm that the rootless setup works by running:

```
docker run hello-world
```

FINN depends on `FINN_XILINX_PATH` and `FINN_XILINX_VERSION` environment variables to point to the Xilinx tools installation directory and version. Append the following to `.bashrc`:

```
FINN_XILINX_PATH=/opt/Xilinx
FINN_XILINX_VERSION=2022.2
```

We provide a fork of the FINN compiler which bundles a modified version of Brevitas:

```
git clone https://github.com/AxterDoesCode/finn.git
```

We can verify that the FINN install works correctly by executing:

```
cd finn
./run-docker.sh quicktest
```

To setup the GPU for use within docker we first need the latest Nvidia `cuda-drivers` package, firstly install `cuda-keyring`:

```
wget https://developer.download.nvidia.com/compute/cuda/repos/$distro/x86_64/
dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update
```

Then install the drivers:

```
sudo apt install cuda-drivers
```

Now to provide GPU access to the FINN docker container, we need to install the Nvidia container toolkit `nvidia-ctk`:

```
sudo apt-get update && sudo apt-get install -y --no-install-recommends \
   ca-certificates \
   curl \
   gnupg2
```

Then add the production reposiitory:

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --d
  && curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidi
    sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-t
    sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

Update the packages list from the repository:

```
sudo apt-get update
```

Finally install the container toolkit packages:

```
export NVIDIA_CONTAINER_TOOLKIT_VERSION=1.19.0-1
  sudo apt-get install -y \
      nvidia-container-toolkit=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
      nvidia-container-toolkit-base=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
      libnvidia-container-tools=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
      libnvidia-container1=${NVIDIA_CONTAINER_TOOLKIT_VERSION}
```

2.1.6 PYNQ

We need to install a PYNQ image to the Zynq-7000 SoC, we use the PYNQ 3.0.1 image pre-built for the Pynq-Z1 which can be obtained from https://www.pynq.io/boards.html. We burn the image to a microSD card using `caligula` (which can be installed with `cargo install caligula`):

```
caligula burn pynq_z1_v3.0.1.img
```

The Arty-Z7-20 needs to have a couple jumpers set to allow booting from the SD card and enabling power from the micro USB port. First we set the JP4/Boot jumper to the SD position by placing the jumper over the top two pins of JP4 as shown in 2.

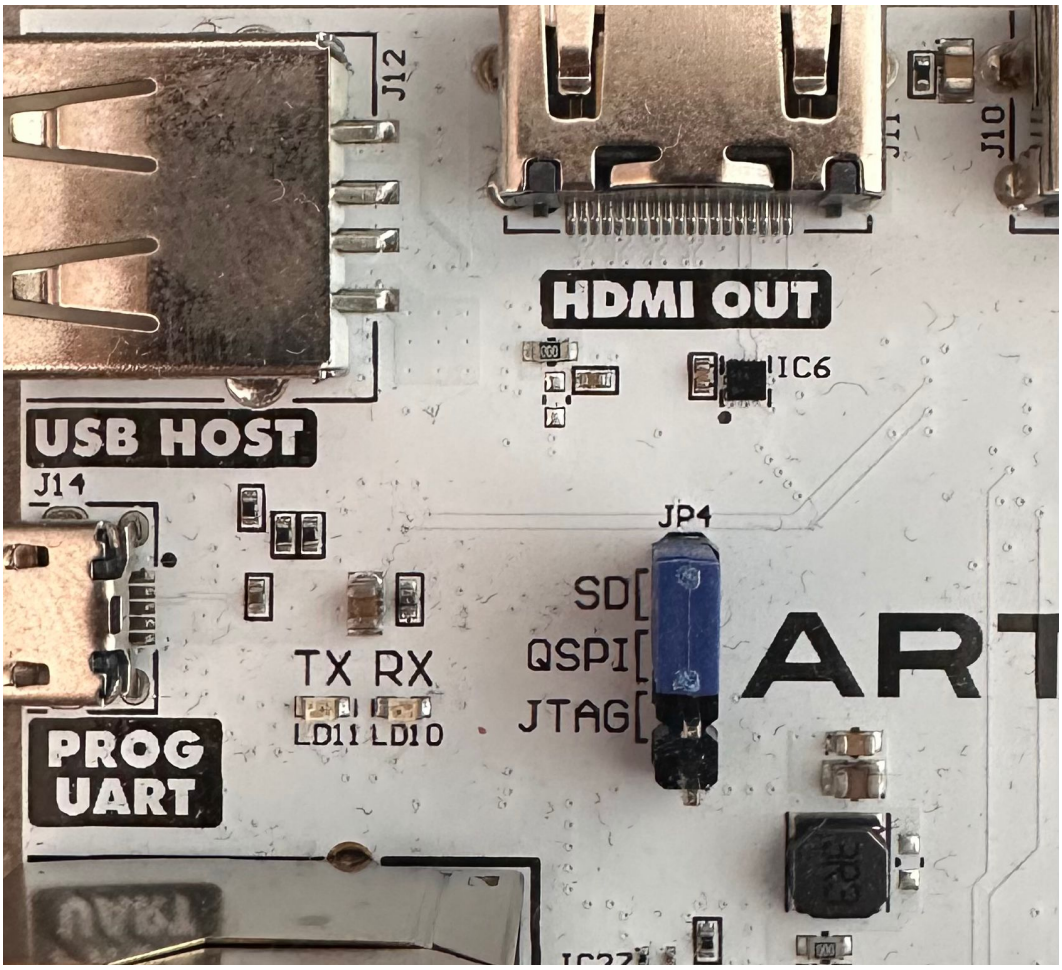

Figure 2: Image of the Arty-Z7-20 JP4 Jumper

Then to power the Arty-Z7-20 from a micro USB cable, set the JP5/Power jumper to the USB position by placing the jumper over the two left pins as shown in 3.

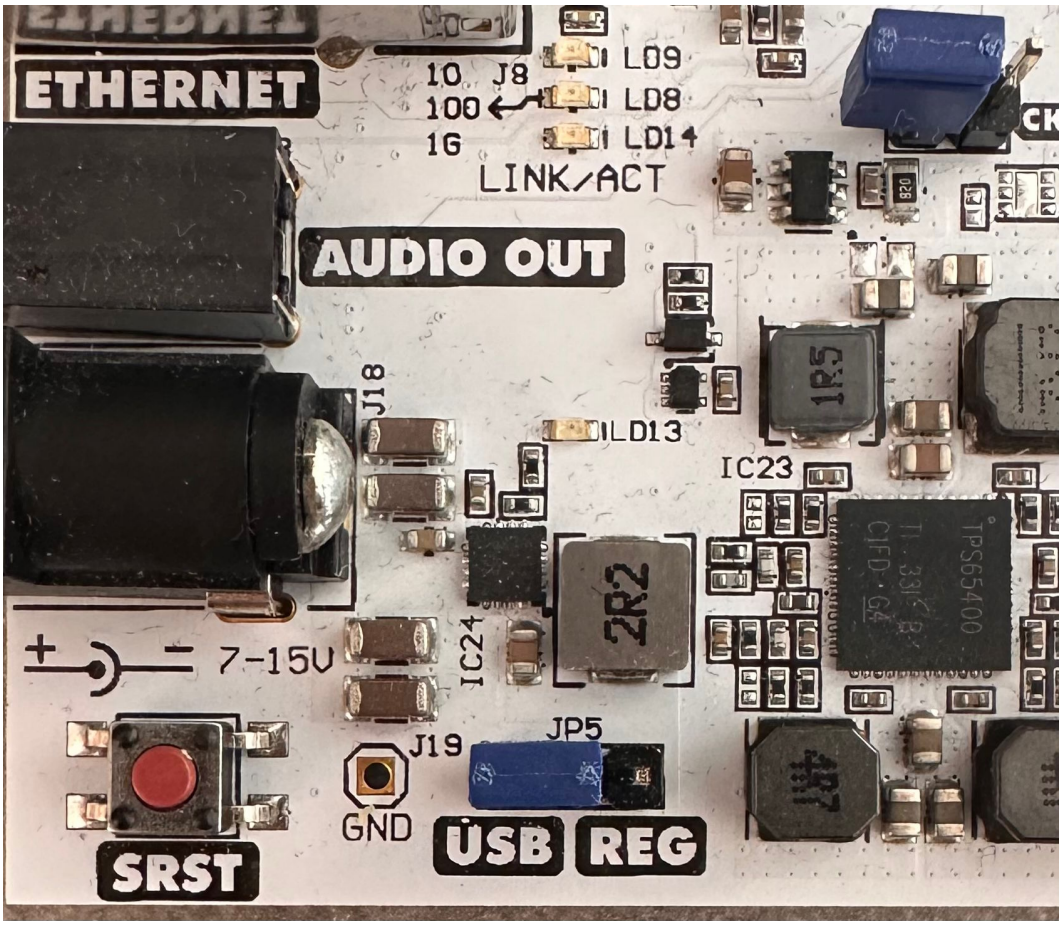

Figure 3: Image of the Arty-Z7-20 JP5 Jumper

Now we can insert the microSD card we flashed with PYNQ, connect the board via ethernet to the Ubuntu device, and power it via micro USB, A successful PYNQ install can be identified by observing LD0-LD3 be completely lit.

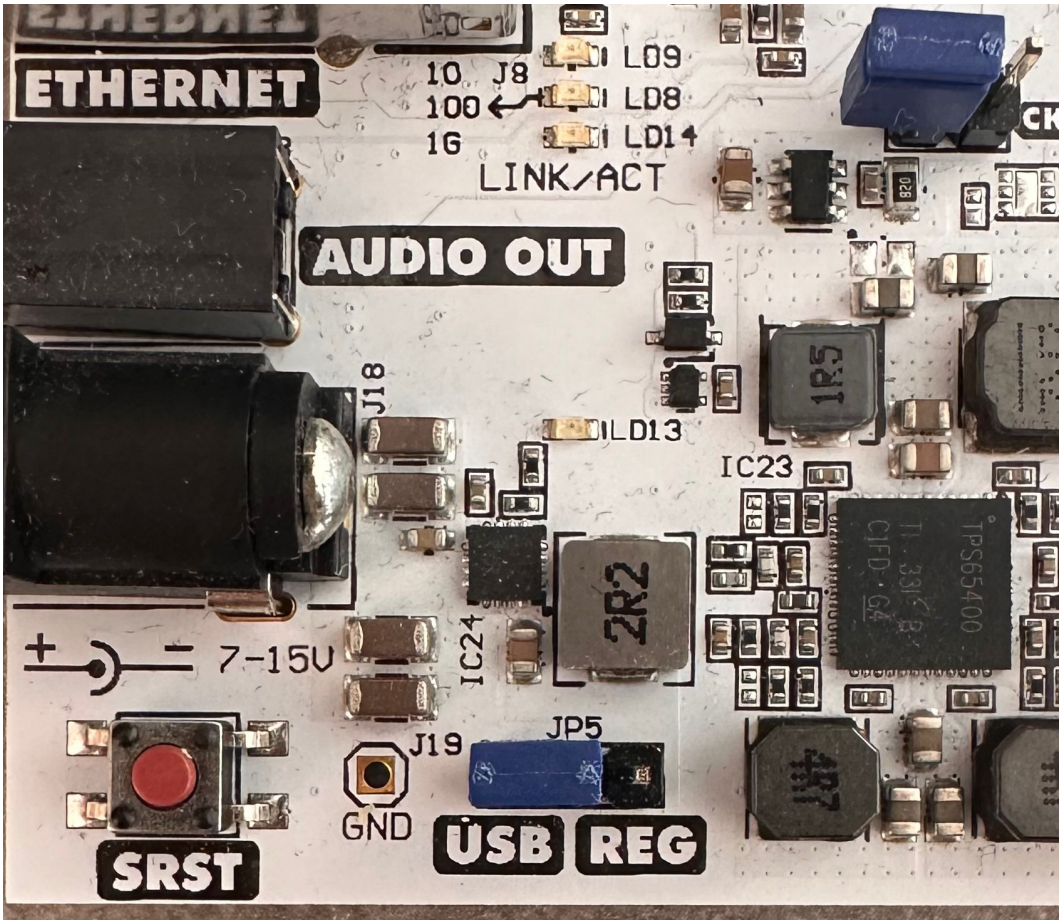

Figure 4: Arty-Z7-20 with successfully flashed PYNQ image

2.2 Training a BNN using Brevitas

2.3 Preparing a trained Brevitas network for hardware synthesis

2.4 Evaluating the BNN compared to running on a GPU // Something like this

3 Experiment 2: Impact of quantisation on neural network FPGA performance

4 Experiment 3: Investigating parallelism in FPGA QNN performance

Basically see if parallelism with certain quantisation sizes has some impact. Use minimal, small, and the w1a1 things in hardware

References

[1] Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 65–74, February 2017. arXiv:1612.07119, doi:10.1145/3020078.3021744.